

BASAVARAJESWARI GROUP OF INSTITUTIONS

BALLARI INSTITUTE OF TECHNOLOGY & MANAGEMENT
Autonomous Institute under VTU, Belagavi | Approved by AICTE, New Delhi Recognized by
Govt. of Karnataka



NACC Accredited Institution*
(Recognized by Govt. of Karnataka, approved by AICTE, New Delhi & Affiliated to
Visvesvaraya Technological University, Belagavi)
"JnanaGangotri" Campus, No.873/2, Ballari-Hospet Road, Allipur,
Ballar1-583 104 (Karnataka) (India)
Ph: 08392 – 237100 / 237190, Fax: 08392 – 237197



DEPARTMENT OF CSE(DATA SCIENCE)

A Mini Project Report On

“Real Time Public Transportation Tracking for Small Cities”

Project Associates:

Priyanka G	3BR23CD071
Sanjana V Choudhari	3BR23CD086
Shirisha T S	3BR23CD088
T Srisha	3BR23CD096

Under the Guidance of

Mr. Azhar Baig M
Asst. Prof
Dept of CSE(DS),
BITM, Ballari.



Visvesvaraya Technological University

Belagavi, Karnataka

2025-2026

BASAVARAJESWARI GROUP OF INSTITUTIONS
BALLARI INSTITUTE OF TECHNOLOGY & MANAGEMENT
*Autonomous Institute under VTU, Belagavi | Approved by AICTE, New Delhi Recognized by
Govt. of Karnataka*



NACC Accredited Institution*
(Recognized by Govt. of Karnataka, approved by AICTE, New Delhi & Affiliated to
Visvesvaraya Technological University, Belgavi)
"JnanaGangotri" Campus, No.873/2, Ballari-Hospet Road, Allipur,
Ballari-583 104 (Karnataka) (India)
Ph: 08392 – 237100 / 237190, Fax: 08392 – 237197



DEPARTMENT OF CSE (DATA SCIENCE)

CERTIFICATE

This is to certify that the project work entitled “**Real Time Public Transportation Tracking for Small Cities**” is a bonafide work carried out by **Priyanka G (3BR23CD071), Sanjana V Choudhari (3BR23CD086), Shirisha TS (3BR23CD088), T Srisha (3BR23CD096)** in partial fulfillment for the award of degree of **Bachelor Degree in CSE(Data Science)** in the VISVESVARAYA TECHNOLOGICAL UNIVERSITY, Belagavi during the academic year 2025-2026. It is certified that all corrections and suggestions indicated for internal assessment have been incorporated in the report deposited in the library. The project has been approved as it satisfies the academic requirements in respect of mini project work prescribed for a Bachelor of Engineering Degree.

Signature of project guide

Mr.Azhar Baig M

Signature of Coordinator

Dr. Jagadish R M

Signature of HoD

Dr. Aradhana D

Name of the Examiner(s)

1)

2)

Signature with Date

Abstract

In many tier-2 cities and small towns, public transportation systems lack real-time digital tracking, leading to uncertainty in bus arrivals, increased waiting time, and reduced reliance on public transport. Passengers often face difficulties in planning their journeys due to the absence of accurate location and arrival information. To address this challenge, this project proposes a **Real-Time Public Transport Tracking System** that enables passengers to track live bus locations, view routes, and receive estimated arrival times through a web-based application.

The system is built using **FastAPI as the backend framework, MongoDB as the database, and React.js for the frontend interface**. Live bus locations are obtained through simulated GPS data and transmitted to users using real-time communication mechanisms. The system supports features such as **bus search by source and destination, dynamic fare estimation based on distance, route visualization, and user authentication**. Additionally, an AI-driven ETA prediction component is introduced to enhance arrival time accuracy using historical and real-time parameters.

This solution improves operational transparency, reduces waiting time, enhances passenger convenience, and promotes the adoption of digital public transport services in small cities. The proposed system is scalable and can be extended to support smart city infrastructure and intelligent transportation applications.

Acknowledgement

The satisfactions that accompany the successful completion of our mini project on **“REAL TIME PUBLIC TRANSPORTATION TRACKING FOR SMALL CITIES”** would be incomplete without the mention of people who made it possible, whose noble gesture, affection, guidance, encouragement and support crowned my efforts with success. It is our privilege to express our gratitude and respect to all those who inspired us in the completion of our mini-project. We are extremely grateful to our Guides **Mr.Azhar Baig M** for their noble gesture, support co-ordination and valuable suggestions given in completing the mini-project. We also thank **Dr.Aradhana D**, H.O.D. Department of CSE(DS), for his co-ordination and valuable suggestions given in completing the mini-project. We also thank the Principal, Management and non-teaching staff for their co-ordination and valuable suggestions given to us in completing project.

Name	USN
PRIYANKA G	3BR23CD071
SANJANA V CHOUDHARI	3BR23CD086
SHIRISHA T S	3BR22CD088
T SRISHA	3BR23CD096

Table of Contents

Chapter No	Chapter Name	Page No
	Abstract	I
	Acknowledgement	II
	Table of Contents	III
	List of Figures	IV
1	Introduction	1
	1.1 Purpose	2
	1.2 Scope of the project	2
	1.3 Problem Statement	2
	1.4 Objectives	2
2	Literature Survey	3-4
3	System Analysis	5
	3.1 Existing System	5
	3.2 Proposed System	5-6
	3.3 System Objectives	6
	3.4 Feasibility Study	6
	3.5 System Requirements	6-7
	3.6 System Architecture Overview	7
	3.7 Expected Outcomes	7
4	System Design	8
	4.1 System Architecture	8
	4.2 Data Flow	9-10
	4.3 Use Case Diagram	10
	4.4 System Modules	11
	4.5 Flow Chart	11-12
	4.6 Database Design	12
5	Implementation	13
	5.1 Development Environment and Technology Stack Configuration	13
	5.2 Hardware Integration and Data Acquisition	13
	5.3 Backend Development(API and Core Logic)	14
	5.4 Database and Caching Implementation	14
	5.5 Frontend Development and Map Visualization	14
	5.6 Deployment Strategy	14-15
	5.7 Integrations	15
6	Testing	16
	6.1 Functional Testing	16
	6.2 UI and Component Testing	16-17

	6.3 Validation and Error and Handling Testing	17
	Output	18-22
	Conclusion	23
	References	24
7	CO-PO Matrix	26-28

List of Figures

Sl.No.	Figure Name	Page No
1	Architecture Diagram	8
2	Data Flow Diagram	10
3	Use Case Diagram	10
4	System Modules Diagram	11
5	Flow Chart	12
6	User Registration Page	18
7	Login Page	19
8	User Dashboard	20
9	Route Search Interface	21
10	Passenger Bus Tracking Interface	22

CHAPTER-1

INTRODUCTION

Public transport forms the backbone of mobility in both metropolitan and non-metropolitan regions. While large metropolitan cities have advanced transport management systems with real time tracking, digital ticketing, and integrated route planning, small cities and semi-urban areas often face severe inefficiencies in their public transport systems. Buses and shared vehicles may not adhere to fixed schedules, real-time visibility is absent, and passengers frequently experience uncertainty about the availability, location, and estimated arrival time of buses. This lack of transparency reduces trust in public transport and pushes commuters to depend on private vehicles, increasing traffic congestion, fuel consumption, and pollution.

The purpose of this project is to design and implement a Real-Time Public Transport Tracking System that addresses these challenges specifically for small cities. The system leverages GPS enabled devices installed in buses and other public transport vehicles, a central server for processing and storing live location data, and a passenger-facing mobile/web application to display real-time information. The solution aims to be cost-effective, scalable, and adaptable to the context of small and medium-sized towns where resources and budgets for sophisticated transport systems are limited.

The proposed system will collect continuous GPS data from buses, transmit it via mobile networks to a backend server, and process it to compute Estimated Time of Arrival (ETA) at bus stops. Passengers can access this information through a web or mobile application, which displays live bus locations on a map, route details, and upcoming arrivals. The project intends to improve the commuting experience by providing timely, reliable information and increasing confidence in public transport usage.

In addition to addressing the information gap for passengers, this system will also support city administrators and transport authorities. Through the backend dashboard, they can monitor fleet operations, detect delays, and plan routes more effectively. Over time, the system can help in gathering valuable data on passenger patterns, demand hotspots, and operational inefficiencies, enabling data-driven decisions for urban transport planning.

This mini-project is significant because it demonstrates the feasibility of developing a low-cost intelligent transport solution tailored for smaller urban centers, where implementing expensive smart city infrastructures is not practical. The project's modular design ensures that it can start small with a few vehicles and later scale to cover the entire city's fleet. Furthermore, it lays the groundwork for future enhancements such as integration with QR-based ticketing, occupancy detection, AI-based delay prediction, and accessibility features for differently-abled passengers.

Real-Time Public Transportation Tracking for Small Cities

1.1 PURPOSE

- The project aims to develop a real-time public transport tracking system for small cities.
- It provides citizens with accurate, live bus locations and Estimated Times of Arrival (ETAs).
- The system enhances public transport convenience, improves reliability, and encourages its use.
- A key purpose is to improve the network's efficiency and support future urban planning initiatives.
- This approach is intended to help reduce traffic congestion and promote sustainable urban mobility.

1.2 SCOPE OF THE PROJECT

- In-scope: Real-time tracking of a fleet of buses, display of bus locations on an interactive map, and ETA calculation.
- Out-of-scope: Integration with payment systems, route optimization algorithms (initial version will use fixed routes), and real-time traffic analysis (initial version will use simplified ETA logic).

1.3 PROBLEM STATEMENT

Uncertainty due to fixed schedules ignoring real traffic forces long, stressful waits, making public transport feel unreliable. This lack of accurate, real-time data drives away riders and prevents managers from instantly fixing service problems.

1.4 OBJECTIVES

- To design and implement a real-time tracking system for public buses.
- To provide citizens with accurate Estimated Times of Arrival (ETAs) for buses at designated stops.
- To create a user-friendly web interface for easy access to information.
- To improve the overall efficiency and reliability of the city's public transport network.
- To provide data for future route optimization and urban planning.

CHAPTER-2

LITERATURE SURVEY

No	Reference	Author	Publisher, year	Key findings	Outcomes	Limitations
1.	Advances in GNSS Positioning and GNSS Remote Sensing	Yasyukevich, Zhang, Venkata R.D	Gade et al. 2024	Highlights improvements in low-cost multi-constellation GNSS systems for higher accuracy, reliability, and resilience against signal interference.	Supports development of cost-effective, accurate, and reliable real-time tracking for public transport in small cities.	Tested in controlled conditions with limited datasets; affected by signal interference and space weather; lacks full real-time transport integration.
2.	IoT in urban development	Salih, M. Sharif, M.Hamdan	PeerJ computer science, 2025	IoT enable real time data transfer,multi-tier architecture improves latency	Provides technical basis for using IoT and hybrid architecture in real time tracking.	Scalability, cost, security, Energy- efficient devices are crucial for IoT
3.	Vehicle tracking system using Greedy forwarding algorithms.	Jimoh	IEEE, 2020	GPS+GSM/GPRS for real time tracking. It uses greedy forward algorithm.	Effective optimized routing algorithms	Algorithm may fail in complex urban roads.Needs continuous network connectivity.

Real-Time Public Transportation Tracking for Small Cities

4.	Public Transport Commuting Analytics: A Longitudinal Study Based on GPS Tracking and Unsupervised Learning	Mariana costa, Alessio DM, Francesco Corman	Springer Nature, 2023	Uses smartphone GPS + AVL (Automatic Vehicle Location) over many trips to analyze commuting behaviour: home/work locations, departure times, line-modes, durations	Helps to understand user travel patterns; useful if you want your tracking to feed into scheduling, route planning, demand estimation	Not primarily about very high accuracy or latency; more about analytics & patterns than real-time control/updates.
5.	Real time bus travel time prediction and reliability quantification	Yuran Sun, James Spall, Wai Wong, Xilei Zhao	Cornell University , 2025	Models link travel time distributions including uncertainty. Outperforms baseline models in prediction accuracy. Provide s interpretability of factors affecting travel time	Helps to predict ETAs accurately and quantify reliability	Needs high quality GTFS-static and real time data. Difficult deployment due to computational complexity.

CHAPTER-3

SYSTEM ANALYS

System analysis involves understanding the transportation challenges in small cities, identifying commuter needs, and determining how the Real-Time Public Transportation Tracking system can solve issues of reliability and congestion using GPS technology. The analysis ensures that the system is scalable, secure, and capable of delivering accurate location updates and ETAs to passengers.

3.1 Existing System

Traditional public transport in small cities relies on fixed schedules that often ignore real-time traffic conditions. Existing issues include:

- **Uncertainty:** Fixed schedules do not account for real-world delays, leading uncertainty.
- **Long Wait Times:** Passengers endure long, stressful waits due to a lack of arrival information.
- **Lack of Data:** Absence of accurate, real-time data prevents managers from fixing service problems instantly.
- **Rider Attrition:** Unreliability drives riders away, potentially increasing personal vehicle use and traffic congestion.

These limitations lead to a perception of unreliability and reduced usage of public transport.

3.2 Proposed System

The proposed **Real-Time Public Transport Tracking System** provides a live monitoring solution specifically tailored for small cities. The system includes:

- **Real-Time Tracking:** Utilizes GPS technology and IoT principles to track bus fleet locations.
- **Interactive Mapping:** Displays live bus movements on a user-friendly map interface.
- **ETA Calculation:** Provides citizens with accurate Estimated Times of Arrival for buses at designated stops.
- **Status Updates:** Offers real-time status information (e.g., "in-transit," "at stop," "delayed").
- **Cross-Platform Access:** Accessible via smartphones, tablets, and computers.

Real-Time Public Transportation Tracking for Small Cities

The system enhances convenience, improves reliability, and promotes sustainable urban mobility.

3.3 System Objectives

- **Real-Time Visibility:** To design and implement a tracking system that displays the real-time location of all active buses.
- **Accurate Information:** To provide citizens with accurate ETAs at designated stops to reduce wait-time stress.
- **User Accessibility:** To create a user-friendly web interface that is easy to navigate for all age groups.
- **Operational Efficiency:** To improve the overall reliability of the city's transport network.
- **Data Collection:** To collect data for future route optimization and urban planning.

3.4 Feasibility Study

A. Technical Feasibility

- **Technologies used:** Node.js, Express.js, WebSockets, MongoDB, and Google Map API.
- **Hardware:** Standard GPS-enabled devices for buses and cloud-based servers (AWS) for backend hosting.
- **Architecture:** The detailed architecture utilizes stream processing and Redis caching, which are established technologies for handling real-time data.

B. Economic Feasibility

- **Cost-Effective Hardware:** The literature survey supports the use of low-cost multi-constellation GNSS systems.
- **Scalable Infrastructure:** Cloud-based hosting allows for pay-as-you-go scaling, avoiding massive upfront infrastructure costs.

C. Operational Feasibility

- **Usability:** The system is designed with an intuitive UI suitable for all age groups.
- **Maintenance:** The system separates data ingestion from user display, allowing for easier maintenance and updates.

3.5 System Requirements

Functional Requirements

Real-Time Public Transportation Tracking for Small Cities

- The system shall display the real-time location of all active buses on a map.
- The system shall calculate and display the ETA for each bus at upcoming stops.
- The system shall allow users to search for specific bus routes and stops.
- The system shall provide bus status updates (e.g., "in-transit," "delayed")

Non-Functional Requirements

- **Latency:** Location data should be updated with minimal latency (target: every 5 seconds).
- **Scalability:** The system must handle an increasing number of buses and users.
- **Security:** Data transmission between devices and the server must be secure.
- **Usability:** The interface must be intuitive for the general public.

3.6 System Architecture Overview

The system consists of:

- **Vehicle Unit:** GPS device sends position, speed, and timestamp via HTTP Ingestion API.
- **Processing Module:** Stream Processing Engine handles map matching and ETA calculation.
- **Storage: MongoDB** stores historical data for analysis and the latest state for quick access.
- **Backend API:** Node.js/Express with WebSockets pushes data to the client.
- **Frontend UI:** React-based interface with Mapbox/Google Maps to display the visual data.

Data flows from the **Bus GPS** → **Ingestion API** → **Stream Processing** → **Database/Cache** → **API Gateway** → **User Dashboard**

3.7 Expected Outcomes

- **Enhanced Convenience:** Citizens have access to live information, reducing wait times.
- **Improved Reliability:** Public transport becomes a more dependable option for daily commuting.
- **Reduced Congestion:** Encourages the use of public transport, thereby reducing traffic congestion.
- **Sustainable Mobility:** Promotes eco-friendly travel habits in small cities.

CHAPTER-4

SYSTEM DESIGN

4.1 System Architecture

The system utilizes a modern client-server architecture powered by cloud infrastructure integration. The design is divided into three primary layers: the **Sensing Layer** (Vehicle Fleet), the **Processing Layer** (Backend Server), and the **Application Layer** (User Interface).

Key Architectural Components:

Vehicle Fleet (Sensing Layer): Equipped with GPS/OBU (On-Board Units) that capture telemetry data including GPS position, speed, and door status.

Real-time Tracking Backend (Processing Layer): A centralized server handling data ingestion, stream processing, and storage. It utilizes an API Gateway to manage traffic and business logic.

Data Routing & Caching: A critical component where data is split between "Hot Storage" (Redis Cache) for immediate retrieval and "Cold Storage" (Time-Series DB) for historical analysis.

User/Passenger App (Application Layer): A web or mobile interface that consumes the API to display bus locations on an interactive map.

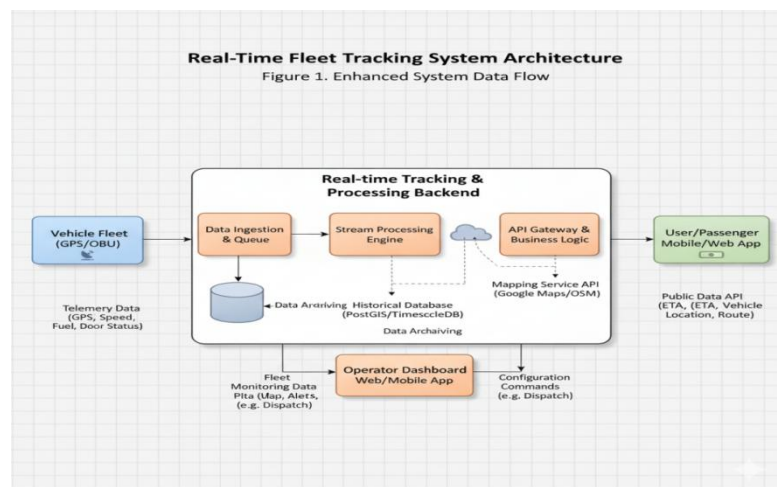


Fig.4.1 Architecture Diagram

4.2 Data Flow

The data flow in the Real-Time Vehicle Tracking System describes how vehicle location information is captured, processed, stored, and delivered to end users.

i. Data Generation (Vehicle Layer)

Each vehicle continuously generates GPS data containing its current position, speed/heading, and timestamp. This raw telemetry is transmitted to the backend.

ii. Data Ingestion (HTTP Ingestion API)

The backend receives the vehicle's data through an HTTP ingestion API. This layer performs validation and converts the incoming stream into a standardized format for further processing.

iii. Real-Time Stream Processing

The validated data is forwarded to the stream processing engine. Here, several real-time operations occur:

- Map matching
- ETA calculation

iv. Data cleaning/filtering

The engine transforms raw GPS readings into meaningful operational information.

v. Data Routing

After processing, the data is routed to two different storage components based on its purpose:

- MongoDB stores the latest state (current location and ETA) for fast retrieval.
- Time-Series Database stores historical telemetry, enabling trend analysis, diagnostics, and performance monitoring.

vi. Application Access through API Gateway

Client applications interact with the system through the API Gateway.

- For real-time requests, the API fetches data from MongoDB.

vii. End-User Consumption

- Passenger Web UI displays live bus positions and ETA.

Real-Time Public Transportation Tracking for Small Cities

- Operator Dashboard uses both real-time and historical data to provide fleet monitoring, alerts, and administrative insights.

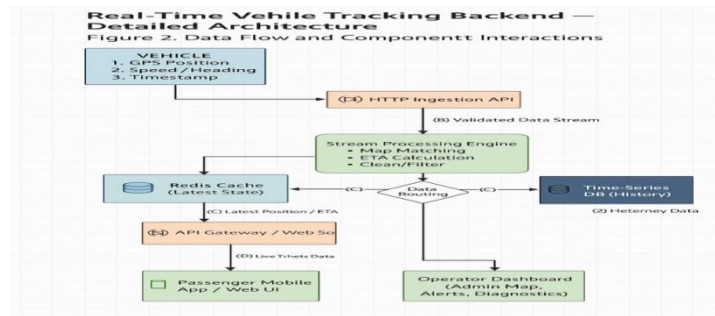


Fig.4.2 Data Flow Diagram

4.3 Use Case Diagram

This diagram illustrates the functional requirements of the Real-Time Public Transport Tracking System and the interactions between primary actors and the system.

Actors:

- User:** Represents the commuter or end-user who interacts with the front-end interface to track buses and plan journeys.
- GPS Data Source:** Represents the external hardware (bus GPS units) responsible for continuously transmitting location data to the system.

Core Use Cases: The system facilitates three primary actions for the User: Viewing Real-time Bus Locations, Searching for Routes/Stops, and Viewing Estimated Time of Arrival (ETA).

System Dependencies: The diagram utilizes an <<include>> relationship to show a critical dependency. Both the "View Real-time Bus Location" and "View ETA for Stops" use cases rely on the "Provide Real-time Location Data" use case. This indicates that the system cannot display locations or calculate ETAs without first successfully receiving data from the GPS Data Source.

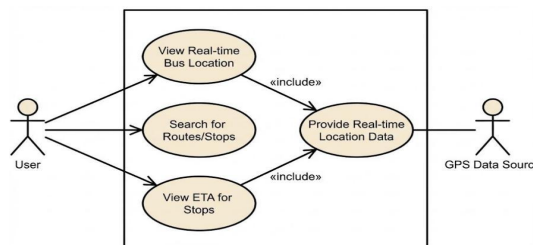


Fig.4.3 Use Case Diagram

4.4 System Modules

Data Ingestion Module: Receives real-time GPS data from bus units and feeds it into the system.

Processing & Storage Module: Processes incoming location data, updates the bus status in the database, and performs ETA calculations.

API Module: Provides endpoints for the front-end to request and retrieve bus location data, routes, and ETAs.

Front-End Module: The user interface, including the interactive map, search bar, and information panels.

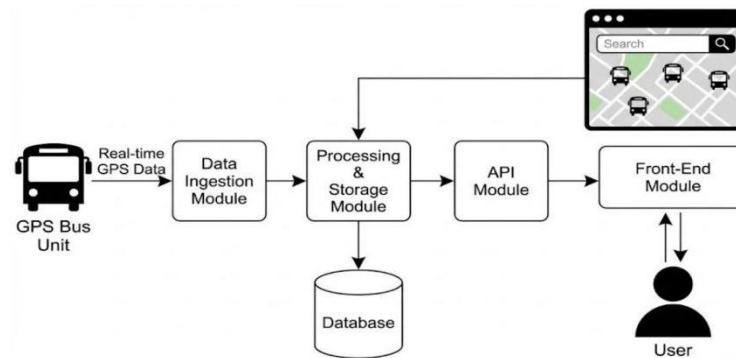


Fig.4.4 System Modules Diagram

4.5 Flow Chart

Flowchart Steps:

- 1) Start
- 2) Bus sends GPS data
- 3) Server receives and validates
- 4) Store data in DB
- 5) Calculate ETA
- 6) API serves live data
- 7) User views map & ETA
- 8) Repeat

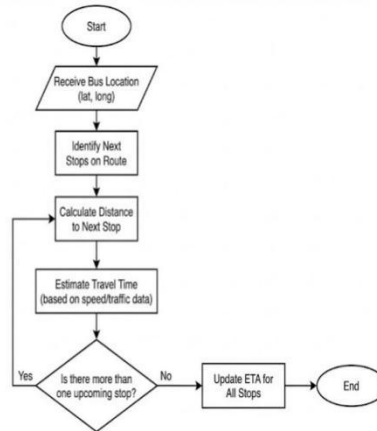


Fig.4.5 Flow Chart

4.6 Database Design

The system employs a hybrid database approach to handle different data types efficiently.

NoSQL Database (MongoDB): Used for storing static and semi-static entities. Based on the project implementation, the collections include:

`users`: Stores user credentials and profile data.

`buses`: Stores bus identifiers (e.g., "Bus B101") and vehicle details.

`routes`: Defines the paths (e.g., "SP Circle to Royal Circle").

`stops`: Contains geo locations of designated bus stops.

`positions`: Stores the last known coordinates of vehicles.

CHAPTER-5

IMPLEMENTATION

This chapter details the practical steps taken to build and integrate the components of the Real-Time Public Transport Tracking system, covering the environment setup, module development, and integration process.

5.1 Development Environment and Technology Stack Configuration

Required Software Installation:

- Node.js and npm (or yarn).
- MongoDB Server and MongoDB Compass (or similar client).
- Development Environment (VS Code, IntelliJ, etc.).

Backend Framework Setup: Initializing the Node.js/Express.js project and configuring dependencies (e.g., Express, Mongoose, body-parser).

Frontend Framework Setup: Initializing the React project (e.g., using Vite) and configuring state management.

5.2 Hardware Integration and Data Acquisition

- **GPS Unit Setup:** Description of the GPS device model (or simulated device) and its configuration parameters.
- **Data Transmission Protocol:** Detailing the protocol used to transmit data from the bus to the server (e.g., HTTP POST requests to the ingestion API or an MQTT/TCP connection).
- **Simulation Strategy :** If real hardware was not used, describe the script or tool used to simulate the movement of multiple buses in real-time.

5.3 Backend Development (API and Core Logic)

- **Data Ingestion API Implementation:** Coding the specific REST endpoint (/api/v1/ingest/location) responsible for receiving and validating raw GPS data.
- **Real-Time Processing Engine:** Implementation of the logic for:
- **Map Matching:** Aligning raw GPS points to defined routes.
- **ETA Calculation Logic:** Defining the algorithm (e.g., historical speed average, distance remaining) used to predict arrival times.
- **Real-Time Communication Setup:** Implementing WebSockets (e.g., using Socket.IO) to push live location updates from the server to the connected frontend clients.

5.4 Database and Caching Implementation

- **MongoDB Schema Creation:** Implementation details for the required collections (routes, stops, buses, positions, users) including field definitions and indexing.
- **Caching Strategy Implementation:** Configuring the Redis Cache (or similar in-memory store) to update the latest position and serve as the primary source for real-time user requests.

5.5 Frontend Development and Map Visualization

- **Map Integration:** Implementation of the interactive map using the chosen API (Mapbox or Google Maps), including API key setup and initialization.
- **Real-Time Display Component:** Developing the React component responsible for maintaining the WebSocket connection and dynamically rendering bus icons and their associated status/ETA on the map.
- **User Interface (UI) Implementation:** Coding the main dashboard, search bar, and login/registration forms.

5.6 Deployment Strategy

- **Hosting Environment:** Description of the cloud platform or local server used (e.g., Heroku, AWS EC2, GCP, or a local machine).
- **Final Deployment Steps:** Procedure for deploying the Node.js backend and the React build for public access.
- **Final URL/Access:** The final link and instructions for accessing the deployed system (e.g., login credentials for demonstration).

5.7 Integrations

- **Google Maps API** for displaying live bus locations and stops.
- **Server-Sent Events (SSE)** for real-time location updates.
- **External journey search APIs** integrated through proxy.
- **Browser Notification API** for ETA and tracking alerts.

CHAPTER-6

TESTING

The Testing and Validation chapter ensures that the developed Real-Time Public Transportation Tracking System meets all specified functional and non-functional requirements. The testing process verifies the accuracy of the location data, the reliability of the real-time updates, and the system's performance under load.

6.1. Functional Testing - Tests whether each feature works correctly.

- **Real-time Location Display**
 - **Test:** An active bus moves along its route.
 - **Result:** The bus icon updates on the map with minimal latency (e.g., every 5 seconds).
 - Ensures the real-time tracking feature works end-to-end as per the reliability requirement.
- **Estimated Time of Arrival (ETA) Calculation**
 - **Test:** User selects a bus stop on a route.
 - **Result:** The system displays a calculated ETA for the next bus at that stop.
 - Confirms the system can accurately calculate and display ETAs.
- **Search for Bus Route/Stop**
 - **Test:** User enters a valid bus route number or stop name in the search bar.
 - **Result:** The map centers on the route/stop, and relevant information (e.g., ETA) is shown in the information panel.
 - Verifies the functionality to search for specific routes and stops.

6.2. UI and Component Testing - Checks if the interface behaves correctly and responds well.

- **Interactive Map Component**
 - **Test:** User zooms in and out of the map or drags the view.
 - **Result:** The map and all bus markers and stop icons render smoothly without errors or delay.

Real-Time Public Transportation Tracking for Small Cities

- Verifies the front-end module and mapping library (e.g., Leaflet.js or Mapbox GL JS) are working correctly.
- **Status Indicators**
 - **Test:** A bus is at a stop, then starts moving, and then is delayed.
 - **Result:** The displayed status updates from "at stop" to "in-transit" to "delayed" on the front-end.
 - Ensures the system accurately communicates bus status to the user interface.

6.3. Validation and Error Handling Testing Ensures the system handles incorrect inputs safely.

- **Stale Data Handling**
 - **Test:** A bus's GPS device stops sending location data for more than the required update interval (e.g., 5 seconds).
 - **Result:** The system displays a clear "GPS Signal Lost" or "Data Stale" status instead of the last known location.
 - Ensures the reliability requirement for data latency is met and users are not shown misleading information.
- **Invalid Route/Stop Search**
 - **Test:** User enters a non-existent route number or stop name into the search bar.
 - **Result:** System shows an error message: "Route/Stop Not Found."
 - Prevents the system from failing when receiving invalid search input.

OUTPUT

New users can create an account by entering their Name, Email, Phone number, and Password. After filling the details, the user can click the Register button to create an account. At the bottom, there is also an option for existing users to log in if they already have an account. This page is a part of the user authentication system that allows secure access to the bus tracking platform.

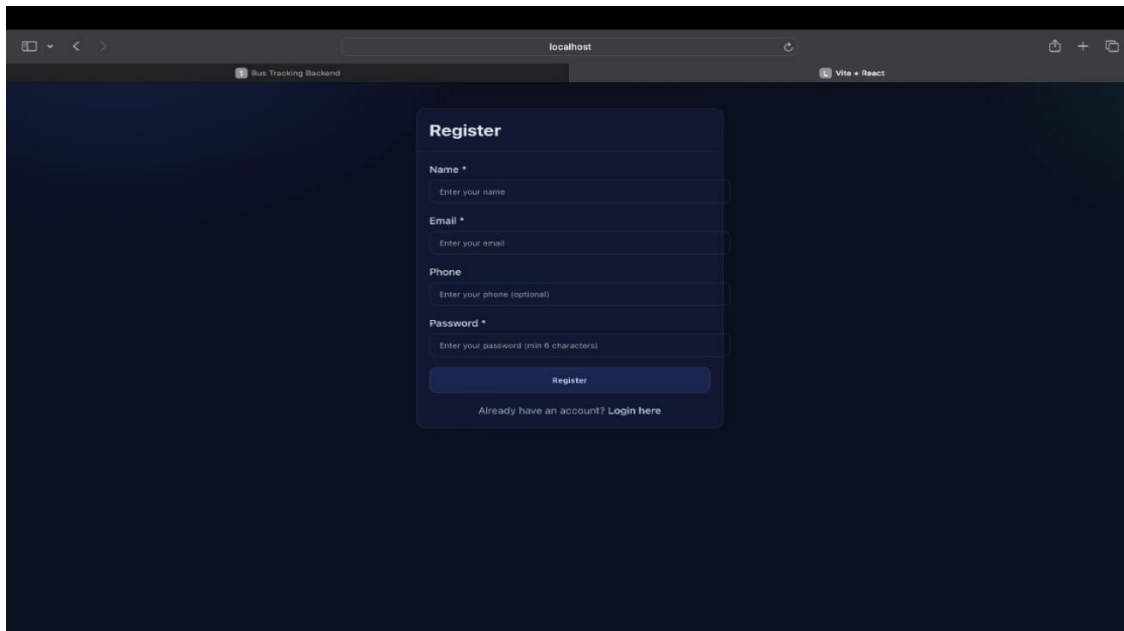
A screenshot of a web browser displaying a user registration form. The browser's address bar shows 'localhost'. The page has a dark blue background. The registration form is a light blue box with the title 'Register'. It contains four input fields: 'Name *' with a placeholder 'Enter your name', 'Email *' with a placeholder 'Enter your email', 'Phone' with a placeholder 'Enter your phone (optional)', and 'Password *' with a placeholder 'Enter your password (min 6 characters)'. Below these fields is a blue 'Register' button. At the bottom of the form, there is a link that says 'Already have an account? Login here'.

Fig 6.1 User Registration Page

Real-Time Public Transportation Tracking for Small Cities

Users can enter their email and password to access the application. A success message confirms that the registration was completed successfully. It ensures secure authentication before using live tracking features.

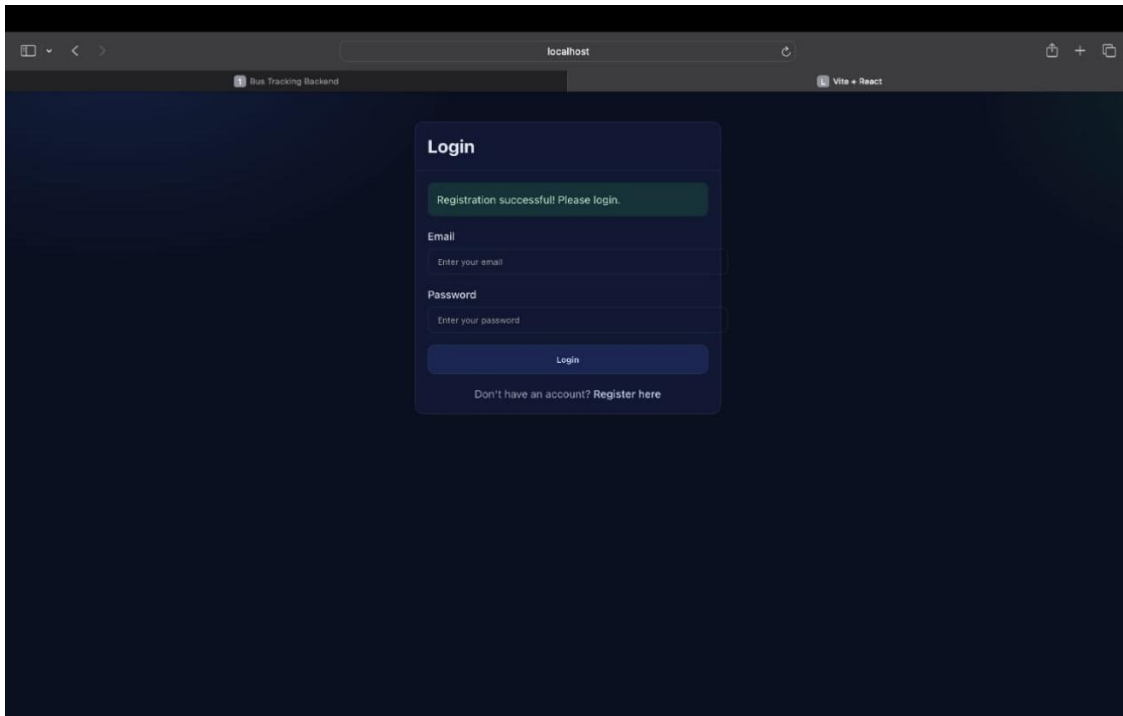


Fig 6.2 Login Page

Real-Time Public Transportation Tracking for Small Cities

The user dashboard of the bus tracking system. It confirms that the user is successfully authenticated and shows their token status. The dashboard provides a clear interface with navigation options like Home and Logout. A quick action button allows the user to proceed to the Bus Tracker page.

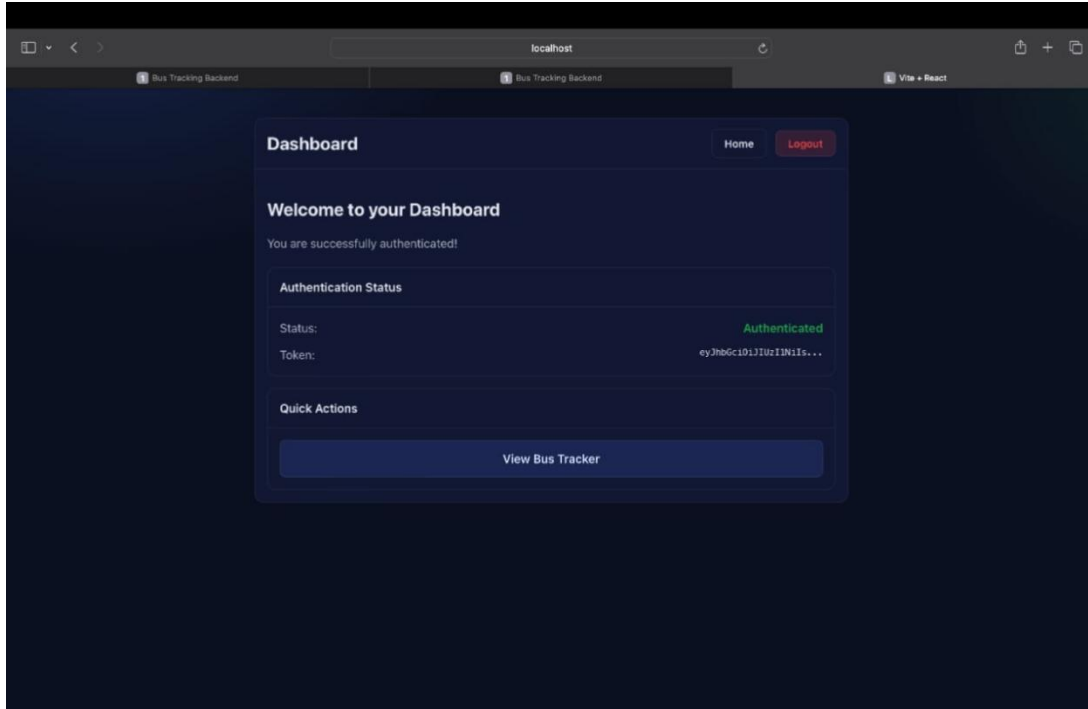


Fig 6.3 User Dashboard

Real-Time Public Transportation Tracking for Small Cities

The map shows the current movement of a selected bus (**Bus B101 – Reverse Route**) with updated markers. The user has searched for a route (SP Circle → Royal Circle), and the system has returned the matching bus service. A **“Stop Tracking”** button appears, indicating that the user is currently tracking the bus in real time. Bus details such as departure, arrival, and fare are displayed for quick reference.

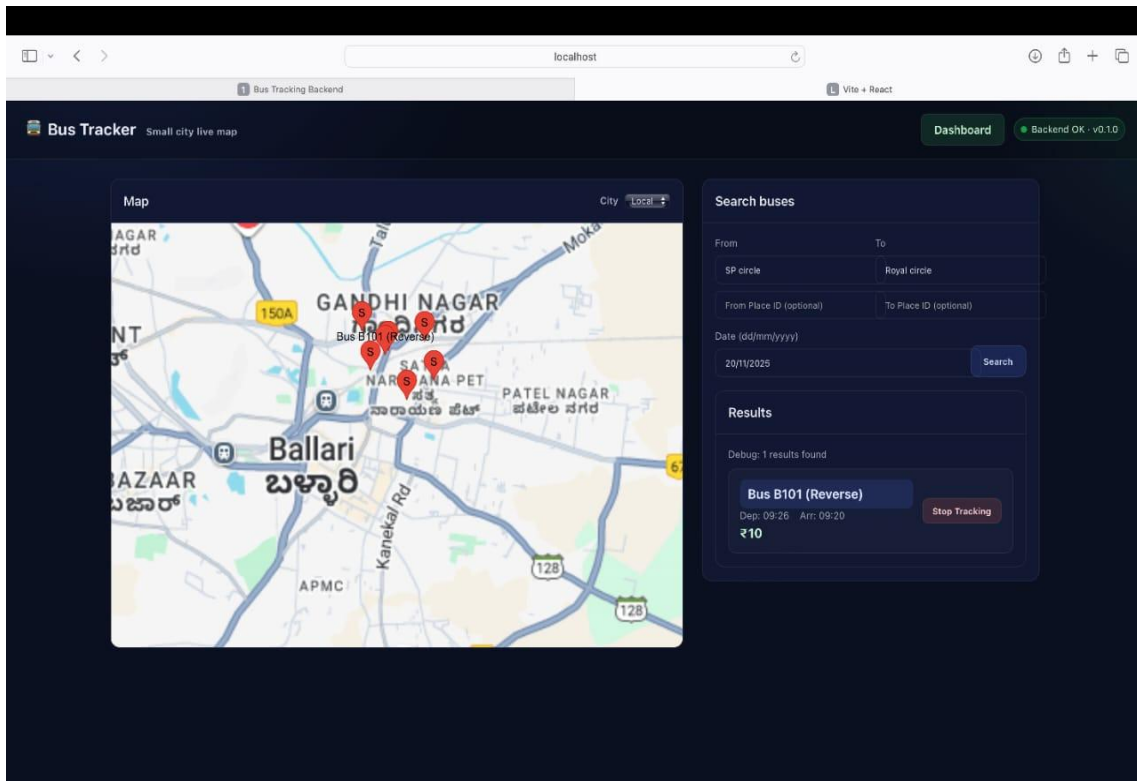


Fig 6.4 Live-Tracking + Route Search Interface

Real-Time Public Transportation Tracking for Small Cities

The map displays the **real-time locations of buses** operating in the city, marked with red map markers. Users can search for available buses by selecting **From**, **To**, and **Date**. The panel on the right shows **search results**, including bus number, departure time, arrival time, and fare. The UI provides a simple, intuitive way for passengers to track buses and plan their travel.

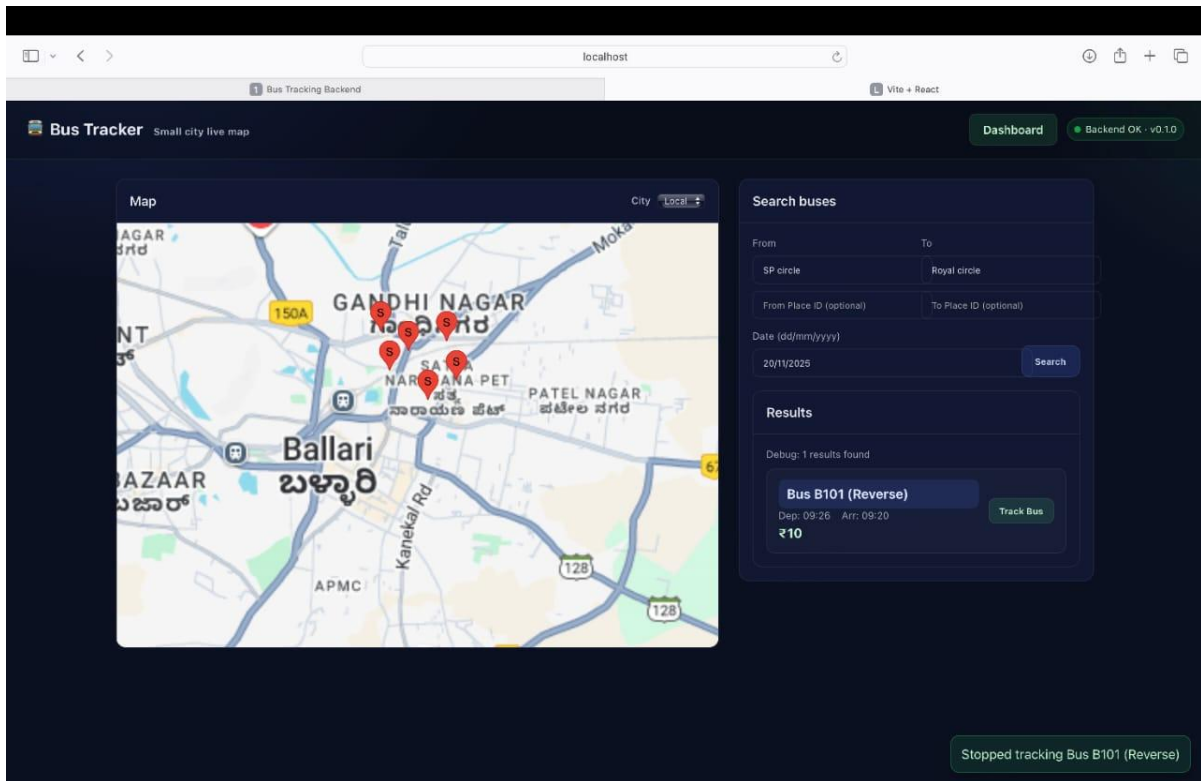


Fig 6.5 Passenger Bus Tracking Interface

CONCLUSION

Public Transport Tracking System tailored for small and semi-urban cities. By combining GPS-based live bus tracking, a web-based frontend, and a modular backend architecture, the system addresses one of the most common problems faced by public transport users—lack of reliable and real-time information about bus locations and arrival times. The solution shows that effective transport intelligence can be achieved even without expensive smart city infrastructure.

The implemented system integrates **Flask and FastAPI backend services, MySQL and MongoDB databases**, and a **React-based frontend** to deliver a complete, end-to-end application. Real-time bus positions are efficiently handled using **Server-Sent Events (SSE)**, allowing live updates to be streamed to multiple users with minimal latency. The use of in-memory storage for live positions ensures fast access and smooth visualization, while persistent storage maintains long-term data such as routes, tickets, schedules, and users.

The frontend application provides an intuitive and modern user interface with interactive maps, route search, authentication, and real-time tracking features. Smooth marker animations, stop-based ETA viewing, and notification support significantly enhance user experience and usability. The authentication system ensures secure access to protected features such as the user dashboard, while the modular design enables easy future expansion.

From an operational perspective, the system also benefits transport administrators by enabling visibility into fleet movement, ticket bookings, and route information. Historical data stored in relational and document databases can later be analyzed to improve scheduling, optimize routes, and enhance service reliability. Although the current ETA calculation uses simple rule-based logic, the architecture is designed to support **future machine learning-based ETA prediction**, making the system forward-compatible.

In conclusion, this project proves the feasibility of developing a **low-cost, scalable, and intelligent public transport tracking solution** suitable for smaller cities. It improves passenger confidence, promotes public transport usage, and lays a strong foundation for future enhancements such as advanced ETA prediction, mobile applications, and integrated ticketing systems. The system is robust enough for real-world pilot deployment and serves as a practical step toward smarter and more sustainable urban mobility.

REFERENCES

- [1] I. Sommerville, *Software Engineering*, 10th ed. Pearson Education, 2016.
- [2] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed. McGraw-Hill Education, 2019.
- [3] Flask Documentation, *Flask: A Lightweight Web Application Framework*, Pallets Projects.
- [4] FastAPI Documentation, *FastAPI: High Performance API Framework*, Tiangolo.
- [5] React Documentation, *React.js Official Documentation*, Meta Platforms.
- [6] MongoDB Documentation, *MongoDB Database Manual*, MongoDB Inc.
- [7] Mozilla Developer Network, *Server-Sent Events (SSE)*.
- [8] Vite Documentation, *Next Generation Frontend Tooling*, Vite Team.

Under the Guidance of



Project Guide:

Mr. Azhar Baig M

Asst. Professor

Dept. of CSE(Data Science)

BITM Ballari

Author's Biography

1. Author's Name: Priyanka G 3BR23CD071



2. Author's Name: Sanjana V Choudhari
3BR23CD086



3. Author's Name: Shirisha T S 3BR23CD088



4. Author's Name: T Srisha 3BR23CD096



CHAPTER-6

CO-PO MAPPING

PROGRAM OUTCOMES:

Sl. No.	Description	POs
1	Engineering Knowledge: Apply the knowledge of mathematics, natural sciences, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop solutions to complex engineering problems.	PO1
2	Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)	PO2
3	Design / Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)	PO3
4	Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis and interpretation of data to provide valid conclusions. (WK8)	PO4
5	Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)	PO5
6	The Engineer and the World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7)	PO6
7	Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)	PO7
8	Individual and Collaborative Team Work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.	PO8
9	Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, and make effective presentations considering cultural, language and learning differences.	PO9

Real-Time Public Transportation Tracking for Small Cities

10	Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.	PO10
11	Life-Long Learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change. (WK8)	PO11

COURSE OUTCOME:

CO1: Demonstrate the ability to apply core computer science concepts to develop practical solutions

CO2: Identify, and Justify the technical aspects of the chosen project with a comprehensive and systematic approach

CO3: Design, code and test software solutions for specific problems

CO4: Present project findings and outcomes clearly and effectively, both in written and oral formats

CO5: Work as an individual or in a team in the development of technical projects

PSO1. Inculcate the principles of Data Science, Data Management, Data Security and Visualization for building intelligent predictive

PSO2. Applying the knowledge of analytics, statistics and Machine Learning concepts to solve real world business problems

Real-Time Public Transportation Tracking for Small Cities

COURSE OUTCOME ASSESSMENT MATRIX:

CO s	Program Outcome's													
	PO 1	PO 2	PO 3	PO 4	PO 5	P O 6	PO 7	PO 8	PO 9	PO1 0	PO1 1	PO1 2	PSO 1	PSO 2
CO 1	2	2	2	-	2	1	-	-	1	-	1	1	2	2
CO 2	2	2	2	2	2	-	-	-	1	-	-	-	2	2
CO 3	2	2	3	2	3	-	-	-	-	-	-	-	3	2
CO 4	-	-	2	-	-	-	-	-	2	3	-	2	-	-
CO 5	-	-	-	-	-	-	-	-	3	3	2	2	-	-